

Java Backend: Roadmap y Checklist de CV

Guía técnica para competir en el mercado (sin "cosas blandas")

Fecha: 1 de febrero de 2026

Qué busca este documento

Que tu CV muestre, en 10 segundos, dos señales duras: (1) ya construiste sistemas backend reales y (2) entendés lo mínimo de producción (tests, base de datos, deploy, CI). El enfoque es Java orientado a roles de Backend Developer (enterprise / integraciones / data-heavy), especialmente útil si querés apuntar a Alemania.

Rol objetivo

- Backend Java Developer (Junior) - Spring Boot, APIs REST, PostgreSQL, Docker, CI/CD.
- Opcionales para diferenciarte: mensajería (Kafka/RabbitMQ) o cloud (AWS/Azure).
- Tu ventaja: experiencia en industria/farma/QA/procesos convertida en productos (trazabilidad, auditoría, workflows).

Checklist obligatorio para competir

Si falta alguno, quedás como "tutorial dev". Si están, ya competís con perfiles de sistemas.

- API REST con Spring Boot (controllers/services/repos, DTOs, validación, manejo de errores coherente).
- Autenticación y autorización (JWT + roles / RBAC).
- Base de datos PostgreSQL con migraciones versionadas (Flyway o Liquibase).
- SQL fuerte (joins, CTEs, índices, transacciones; nociones de performance con EXPLAIN).
- Testing: JUnit + Mockito + integración (ideal: Testcontainers).
- Docker: Dockerfile + docker-compose (app + db) con "un comando y corre".
- CI/CD: GitHub Actions (build + test + lint; opcional deploy).

Cómo se evidencia en el CV (tabla rápida)

Elemento	Evidencia técnica	Dónde se muestra
Spring Boot	Endpoints REST, DTOs, Swagger/OpenAPI	Sección Proyectos + link GitHub
Auth (JWT/RBAC)	Login, roles, filtros por permiso	README + endpoints protegidos

PostgreSQL + migraciones	Flyway/Liquibase, esquema, constraints	Repo (carpeta migrations)
Testing	JUnit, Mockito, Testcontainers, cobertura razonable	Badge CI + carpeta tests
Docker + CI	Dockerfile, docker-compose, GitHub Actions	README + workflows
Diferencial	Kafka/Rabbit o Cloud deploy	Proyecto 2 o módulo extra

Roadmap de aprendizaje (prioridad real)

Orden pensado para maximizar empleabilidad: primero lo que te da entrevistas, después lo que te diferencia. Los tiempos son orientativos; lo importante es entregar evidencias (proyectos).

Etapas 1 - Java core (base)

- OOP bien (clases, interfaces, composición), excepciones, colecciones, generics, streams.
- Build tool: Maven o Gradle.
- Estándar recomendado: Java 17+.
- Entregable: mini app + tests básicos + README.

Etapas 2 - Spring Boot (backend real)

- REST: recursos, status codes, paginación/filtrado, validaciones.
- Persistencia: JPA/Hibernate; DTOs; mapeo limpio; manejo de errores (ControllerAdvice).
- Documentación: OpenAPI/Swagger.
- Seguridad: JWT + roles (RBAC).
- Entregable: API lista para demo (Swagger, auth, logs).

Etapas 3 - SQL y PostgreSQL (diferencial rápido)

- Modelado relacional: constraints, foreign keys, unique, índices.
- Transacciones e isolation levels (noción práctica).
- Optimización: EXPLAIN, índices, consultas pesadas.
- Migraciones: Flyway/Liquibase.

Etapas 4 - Testing serio

- Unit tests: JUnit 5 + Mockito.
- Integration tests: Testcontainers (Postgres en contenedor).
- API tests (happy path + edge cases).

Etapas 5 - Docker + CI/CD

- Dockerfile + docker-compose (app + db).
- GitHub Actions: build, test, lint, ejecutar tests de integración.
- Entregable: badge de CI + instrucciones claras de ejecución.

Etapas 6 - Cloud (elegí 1)

- AWS o Azure: deploy de API + base managed + secrets/config + logs.
- Entregable: URL pública + guía de deploy en README.

Etapas 7 - Diferencial enterprise (elegí 1)

- Mensajería/eventos: Kafka o RabbitMQ (producer/consumer, retries, idempotencia).
- Seguridad aplicada: OWASP Top 10 + rate limiting + hardening básico.
- Data engineering liviano: jobs batch, calidad de datos, pipelines simples.

Proyectos recomendados (para que tu CV pese)

Hacé 2 proyectos grandes (bien terminados) en lugar de muchos chicos. Tu carrera y tu trabajo en farma/industria son una ventaja si los convertís en features técnicas.

Proyecto 1 (principal): QMS / Control de Cambios y Documentos (GxP-like)

- Workflow: borrador -> revisión -> aprobado (y opcional: rechazado).
- Auditoría completa: quién, cuándo, qué cambió (historial inmutable o tabla de auditoría).
- Roles: admin, revisor, autor (RBAC).
- Trazabilidad: eventos por estado, comentarios, adjuntos opcionales.
- Export: CSV (PDF opcional).
- Stack obligatorio: Spring Boot + PostgreSQL + Flyway/Liquibase + JWT + tests + Docker + CI + OpenAPI.

Proyecto 2 (diferencial): procesamiento por eventos

- API recibe un evento (ej: 'cambio creado'), lo publica a Kafka/Rabbit.
- Worker consume, procesa, guarda trazabilidad y resultados.
- Reintentos y manejo de fallas; idempotencia para evitar duplicados.
- Observabilidad mínima: logs + health + métricas básicas (opcional).

Cómo escribir estos proyectos en el CV (ejemplo)

QMS / Change Control Platform - Java 17, Spring Boot, PostgreSQL, Flyway, JWT RBAC, OpenAPI, JUnit/Mockito, Testcontainers, Docker Compose, GitHub Actions. Implementé workflows de aprobación, auditoría y trazabilidad; validaciones, paginación y manejo de errores consistente.

Tu experiencia (Ing. Química) convertida en valor técnico

No escondas tu perfil: usalo para diferenciarte. En el CV, traducí tu trabajo a competencias técnicas y de dominio (trazabilidad, compliance, gestión documental, control de cambios).

Ejemplos de bullets "técnicos" para tu experiencia actual

- Automatización de flujos de gestión documental y control de cambios (VBA / Power Automate) con validaciones y trazabilidad.
- Estandarización y control de versiones de documentación operativa con foco en auditoría y cumplimiento.
- Gestión y análisis de datos operativos para seguimiento de estados, tiempos y aprobaciones (reporting y consistencia).

Checklist final antes de postular

- En la primera mitad de la primera página del CV aparecen: Java + Spring Boot + PostgreSQL + Docker + CI + Testing.
- Tenés 2 links de proyectos (GitHub) que corren con docker-compose y tienen README profesional.
- Los tests corren en CI y hay badge visible.
- Hay autenticación (JWT) y roles implementados (no solo mencionado).
- Al menos un diferencial: Kafka/Rabbit o Cloud deploy (real, con evidencia).
- Cada proyecto tiene 3-5 bullets técnicos medibles (features y stack), no texto genérico.